```
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
#include <stdexcept>
#include <type_traits>
using namespace std;

// Constants
constexpr long long PAISE_PER_RUPEE = 100;
constexpr long long MINIMUM_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;

// Zone data type
enum class Zone {
    Inner,
    Middle,
    Outer
};

// Passenger data
struct Passenger {
    string passengerId;
    string name;
    bool isStudent;
    bool isSeniorCitizen;
};

// Journey data
struct Journey {
```

```
    long long journeyId;
    Zone zone;
    int hour;
    int minute;
};

// Fare table using integer paise
struct FareTable {
    long long innerFare = 10 * PAISE_PER_RUPEE;
    long long middleFare = 20 * PAISE_PER_RUPEE;
    long long outerFare = 30 * PAISE_PER_RUPEE;

    long long getFare(Zone zone) const {
        switch (zone) {
            case Zone::Inner:
                return innerFare;
            case Zone::Middle:
                return middleFare;
            case Zone::Outer:
                return outerFare;
        }

        throw invalid_argument("Invalid zone");
    }
};

// Smart card
class SmartCard {
private:
    string cardId;
```

```
    long long balancePaise;

public:
    SmartCard(string id, long long balance)
        : cardId(id), balancePaise(balance) {
        if (balance < 0) {
            throw invalid_argument("Balance cannot be negative");
        }
    }

    long long getBalance() const {
        return balancePaise;
    }

    bool deductFare(long long farePaise) {
        if (farePaise < 0 ||
            balancePaise - farePaise < MINIMUM_BALANCE_PAISE) {
            return false;
        }

        balancePaise -= farePaise;
        return true;
    }

    string getCardId() const {
        return cardId;
    }
};

// Check peak hours
```

```
bool isPeakHour(int hour, int minute) {
   if (hour < 0 || hour > 23 ||
      minute < 0 || minute > 59) {
      throw invalid_argument("Invalid travel time");
   }

   int totalMinutes = hour * 60 + minute;

   bool morningPeak =
      totalMinutes >= 480 && totalMinutes < 600;

   bool eveningPeak =
      totalMinutes >= 1020 && totalMinutes < 1200;

   return morningPeak || eveningPeak;
}

// Calculate fare in integer paise
long long calculateFare(
   const FareTable& table,
   Zone zone,
   const Passenger& passenger,
   int hour,
   int minute
) {
   long long farePaise = table.getFare(zone);

   if (isPeakHour(hour, minute)) {
      farePaise = farePaise *
            (100 + PEAK_SURCHARGE_PERCENT) / 100;
```

```
    }

    int concession = 0;

    if (passenger.isStudent || passenger.isSeniorCitizen) {
        concession = 50; // Assumed concession
    }

    farePaise = farePaise * (100 - concession) / 100;

    return farePaise;
}

// Safe balance addition
long long addBalance(long long balance, long long amount) {
    if (amount < 0 ||
        balance > numeric_limits<long long>::max() - amount) {
        throw overflow_error("Invalid amount or balance overflow");
    }

    return balance + amount;
}

int main() {
    try {
        FareTable fareTable;

        Passenger passenger = {
            "STU101", "Student", true, false
        };
```

```
Journey journey = {
   1001, Zone::Middle, 8, 30
};

SmartCard card("CARD101", 10000);

long long farePaise = calculateFare(
   fareTable,
   journey.zone,
   passenger,
   journey.hour,
   journey.minute
);

cout << "NAMMA METRO DATA TYPE VALIDATION\n";
cout << "Card ID: " << card.getCardId() << '\n';
cout << "Passenger ID: " << passenger.passengerId << '\n';
cout << "Journey ID: " << journey.journeyId << '\n';

cout << fixed << setprecision(2);

cout << "Initial Balance: Rs. "
    << card.getBalance() / 100.0 << '\n';

cout << "Calculated Fare: Rs. "
    << farePaise / 100.0 << '\n';

if (card.deductFare(farePaise)) {
   cout << "Journey successful.\n";
```

```
} else {
   cout << "Journey rejected: minimum balance "
        << "of Rs. 50 must remain.\n";
}

cout << "Remaining Balance: Rs. "
    << card.getBalance() / 100.0 << '\n';

// Demonstrate safe integer addition
long long collectionPaise = 0;
collectionPaise = addBalance(collectionPaise, farePaise);

cout << "Collection after one journey: Rs. "
    << collectionPaise / 100.0 << '\n';

// Demonstrate type compatibility
int zoneNumber = static_cast<int>(Zone::Middle);
bool validBalance = card.getBalance() >= MINIMUM_BALANCE_PAISE;

cout << "Zone enum converted explicitly: "
    << zoneNumber << '\n';
cout << "Minimum balance rule satisfied: "
    << boolalpha << validBalance << '\n';

cout << "\nDATA TYPE SIZES\n";
cout << "bool: " << sizeof(bool) << " byte(s)\n";
cout << "char: " << sizeof(char) << " byte(s)\n";
cout << "int: " << sizeof(int) << " byte(s)\n";
cout << "long long: " << sizeof(long long) << " byte(s)\n";
cout << "float: " << sizeof(float) << " byte(s)\n";
```

```
        cout << "double: " << sizeof(double) << " byte(s)\n";

        cout << "long long maximum: "
             << numeric_limits<long long>::max() << '\n';

        // Compare double and integer money representation
        double doubleFare = 0.1 + 0.2;
        cout << setprecision(17);
        cout << "Double precision example (0.1 + 0.2): "
             << doubleFare << '\n';

        cout << setprecision(2);
        cout << "Integer paise example (10 + 20 paise): "
             << (10 + 20) / 100.0 << '\n';
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```